Laboratorio Práctico - Callback Asíncrono (30 minutos)

Objetivo del Ejercicio:

Después de completar este laboratorio, podrás:

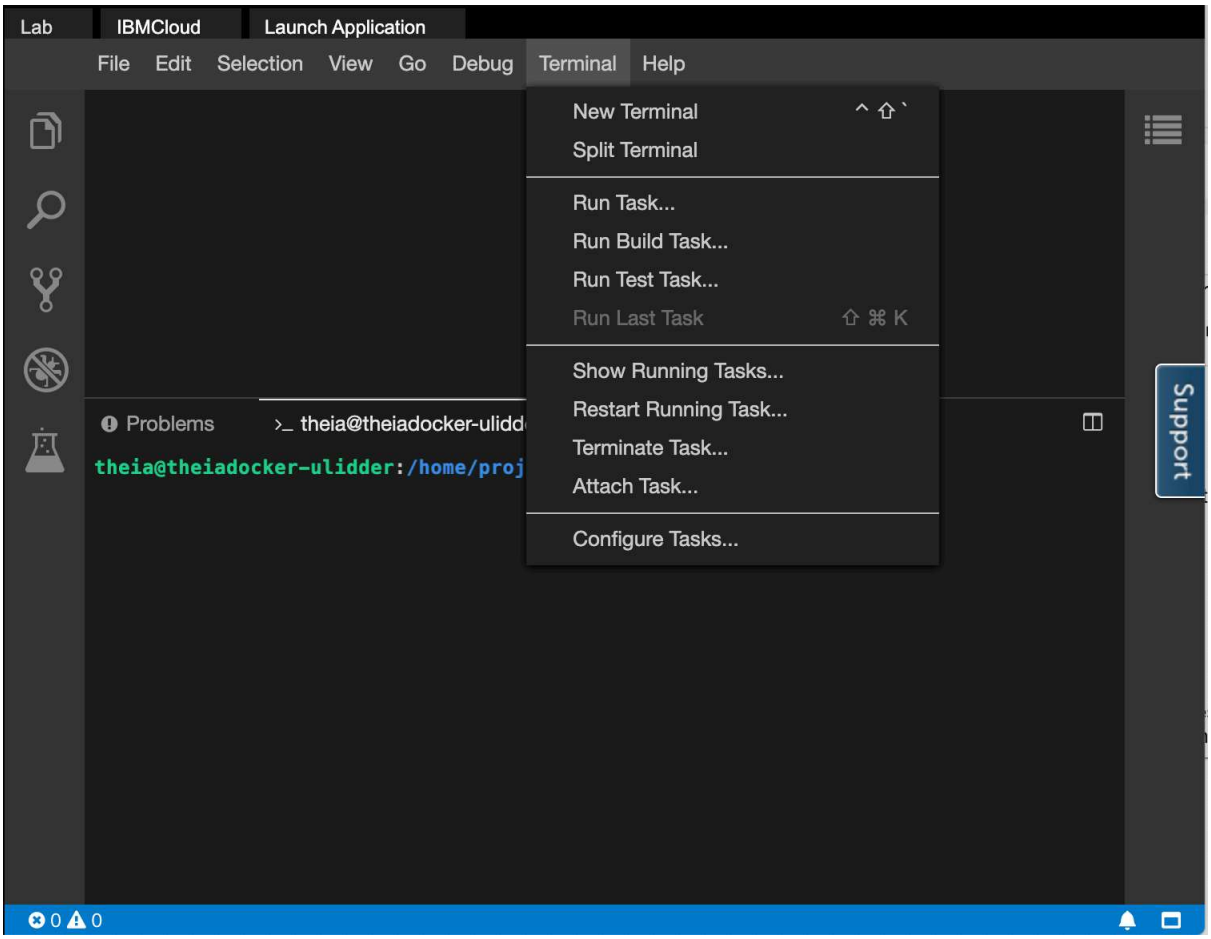
- Describir callbacks asíncronos
- Crear una aplicación en Node.js

Requisitos Previos

- Conocimientos básicos de JavaScript

Tarea 1 : Configuración - Verificar los archivos clonados

1. Abre una ventana de terminal utilizando el menú en el editor: Terminal > Nueva Terminal.



2. Cambia a tu carpeta de proyecto.

```
cd /home/project
```

3. Verifica si tienes la carpeta **lkpho-Cloud-applications-with-Node.js-and-React**

```
ls /home/project
```

Si lo haces, puedes saltar al paso 5.

4. Clona el repositorio git que contiene los artefactos necesarios para este laboratorio, si aún no existe.

```
git clone https://github.com/ibm-developer-skills-network/lkpho-Cloud-applications-with-Node.js-and-React.git
```

5. Cambia al directorio para este laboratorio.

```
cd lkpho-Cloud-applications-with-Node.js-and-React/CD220Labs/async_callback/
```

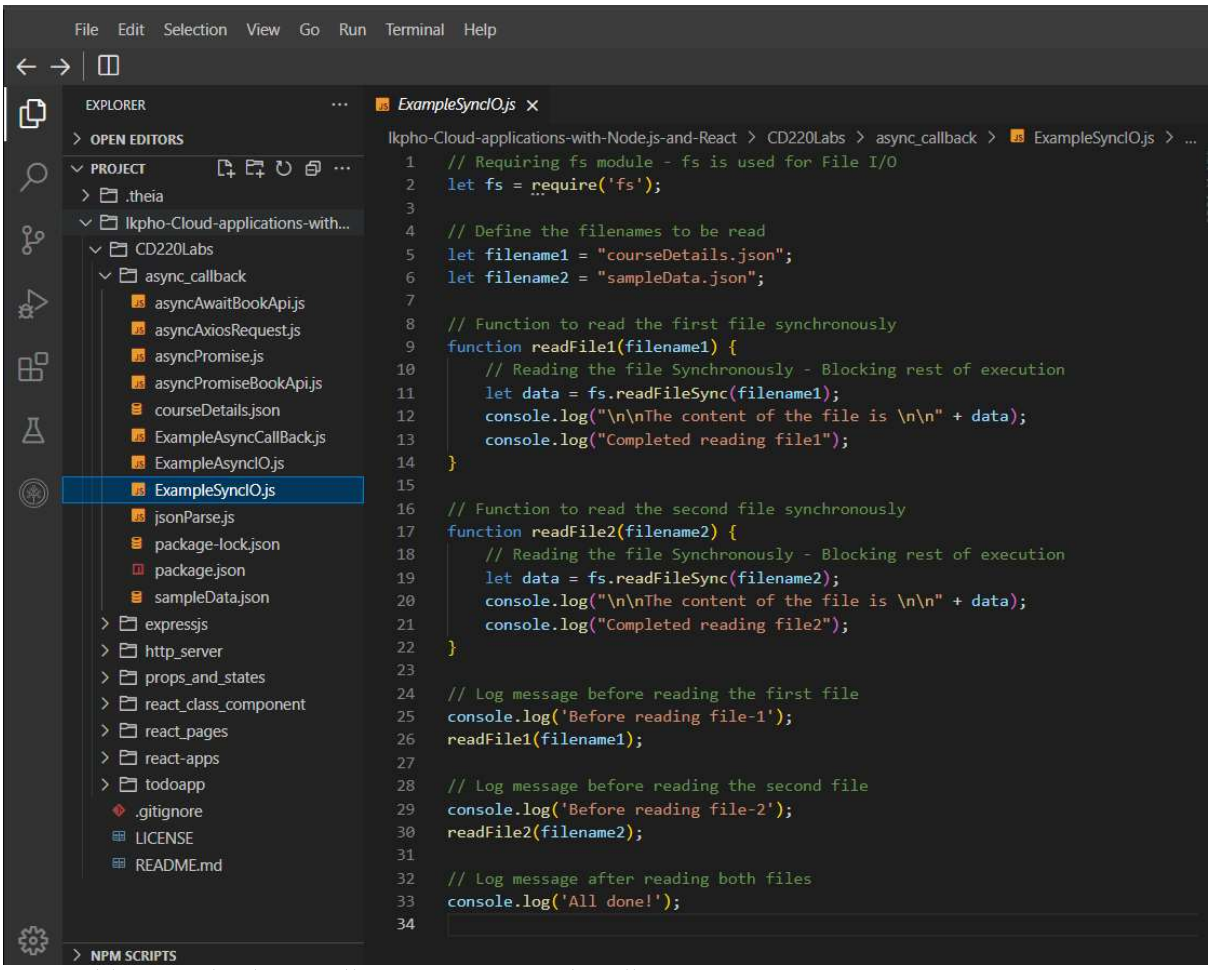
6. Enumera el contenido de este directorio para ver los artefactos de este laboratorio.

```
ls
```

Debes tener algunos archivos de ejercicio que estarás ejecutando en los ejercicios.

Tarea 2: I/O Sincrónico

En la vista del explorador de archivos ExampleSyncIO.js. Aparecería así.



▶ También puedes hacer clic aquí para ver el código

Aquí hay una explicación del código en él:

- `require('fs')` importa el módulo fs incorporado de Node.js, que proporciona funcionalidad relacionada con el sistema de archivos.
- `readFile1(filename1)` y `readFile2(filename2)` definen funciones para leer `courseDetails.json` y `sampleData.json` respectivamente de manera sincrónica utilizando `fs.readFileSync`, que bloquea la ejecución posterior hasta que se lee cada archivo.
- Antes de llamar a las funciones `readFile1(filename1)` y `readFile2(filename2)`, se imprimen mensajes en el registro para indicar el inicio del proceso de lectura para cada archivo.
- Después de iniciar la lectura de ambos archivos, se imprime un mensaje final en el registro "¡Todo listo!" para indicar que el código se ha ejecutado más allá de la iniciación de la lectura de archivos.

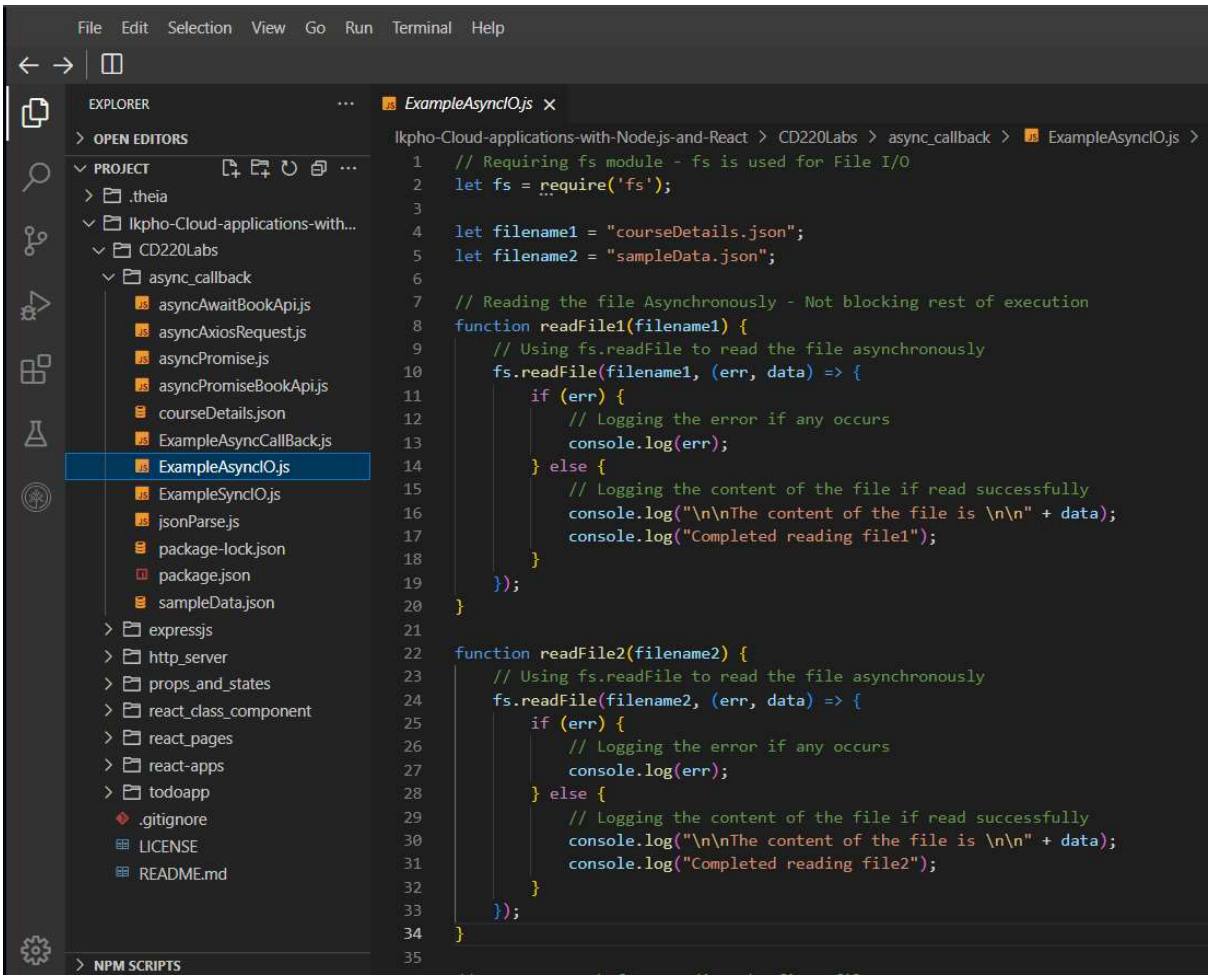
2. En la ventana del terminal, ejecuta el servidor con el siguiente comando.

```
node ExampleSyncIO.js
```

Observe que los dos archivos se están leyendo de manera síncrona, uno después del otro. Esto será evidente por el orden en que aparece el registro en la consola.

Tarea 3: IO Asincrónico

1. En la vista del explorador de archivos ExampleAsyncIO.js. Aparecería así.



▶ También puedes hacer clic aquí para ver el código

Explicación del Código

- `require('fs')` importa el módulo fs integrado de Node.js, que proporciona funcionalidad relacionada con el sistema de archivos.

- `readFile1(filename1)` y `readFile2(filename2)` definen funciones para leer `courseDetails.json` y `sampleData.json` respectivamente utilizando `fs.readFile`, que lee los archivos de manera asincrónica, permitiendo que el resto del código se ejecute sin esperar a que la lectura del archivo se complete.

- En cada función, `fs.readFile(filename, callback)` lee el archivo especificado. Si ocurre un error durante la lectura, se registra el error; de lo contrario, se registra el contenido del archivo y un mensaje de finalización.

- Antes de llamar a las funciones `readFile1(filename1)` y `readFile2(filename2)`, se imprimen mensajes de registro para indicar el inicio del proceso de lectura para cada archivo.

- Después de iniciar la lectura de ambos archivos, se imprime un mensaje de registro final “¡Todo listo!” para indicar que el código se ha ejecutado más allá de la iniciación de la lectura del archivo.

2. En la ventana del terminal, ejecuta el servidor con el siguiente comando.

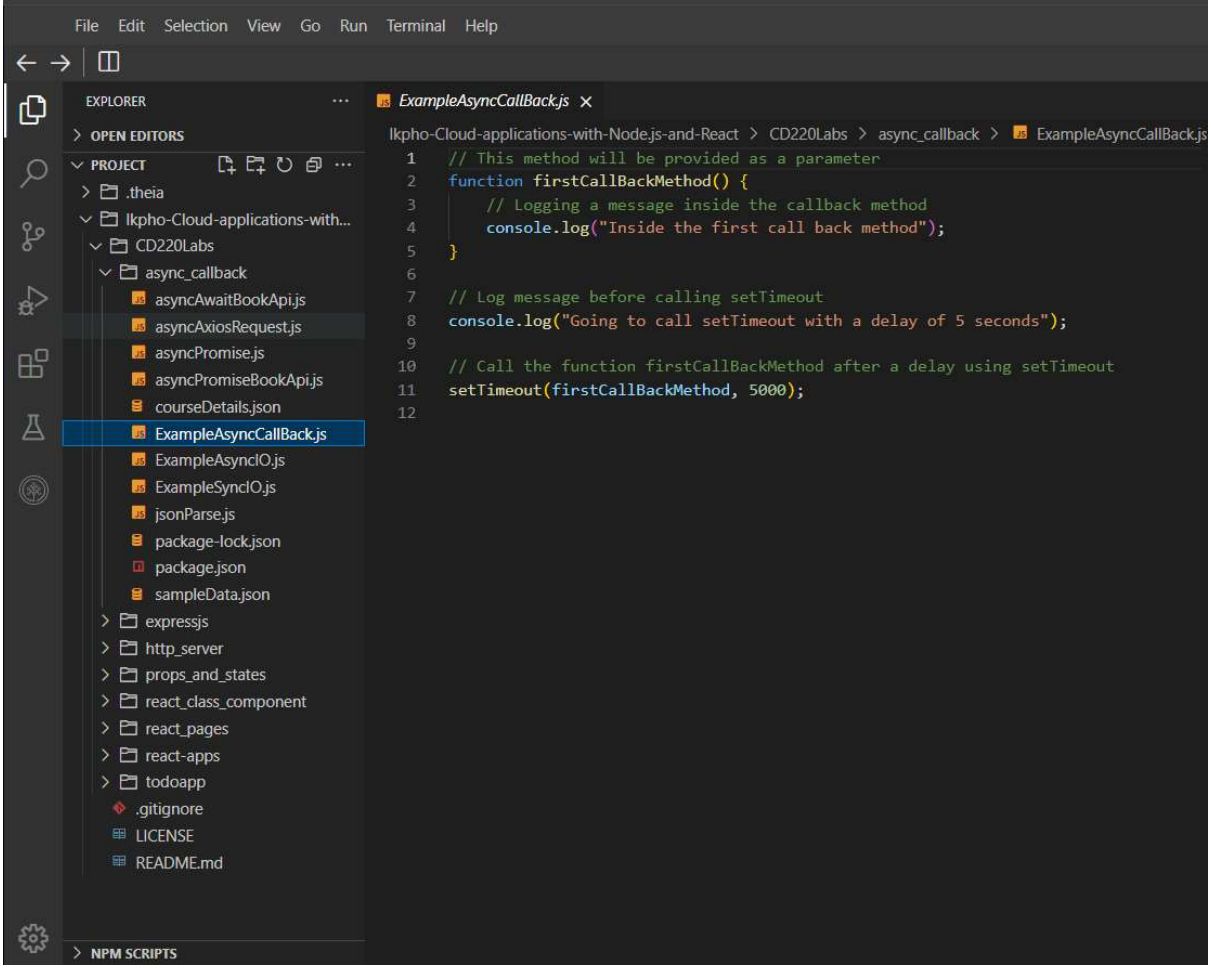
```
node ExampleAsyncIO.js
```

Observe que los dos archivos se están leyendo de manera asíncrona. Esto será evidente por el orden en que aparece el registro en la consola. Los siguientes tres registros en la consola aparecen antes de que se imprima el contenido del archivo, aunque los registros se llaman en el código en un orden diferente.

```
Before the reading the file-1
Before the reading the file-2
All done!
```

Tarea 4: Creando Funciones de Callback

1. En la vista del explorador de archivos ExampleAsyncCallBack.js. Se vería así.



▶ También puedes hacer clic aquí para ver el código

Explicación del Código

- `firstCallBackMethod()` es una función que registra un mensaje “Dentro del primer método de retorno”. Esta función se utilizará como un callback.

- Antes de llamar a `setTimeout`, se imprime un mensaje de registro “Se va a llamar a setTimeout con un retraso de 5 segundos” para indicar la próxima configuración del temporizador.

- `setTimeout(firstCallBackMethod, 5000)` programa la ejecución de `firstCallBackMethod()` después de un retraso de 5000 milisegundos (5 segundos), permitiendo que el resto del código continúe ejecutándose de inmediato mientras el temporizador se ejecuta en segundo plano.

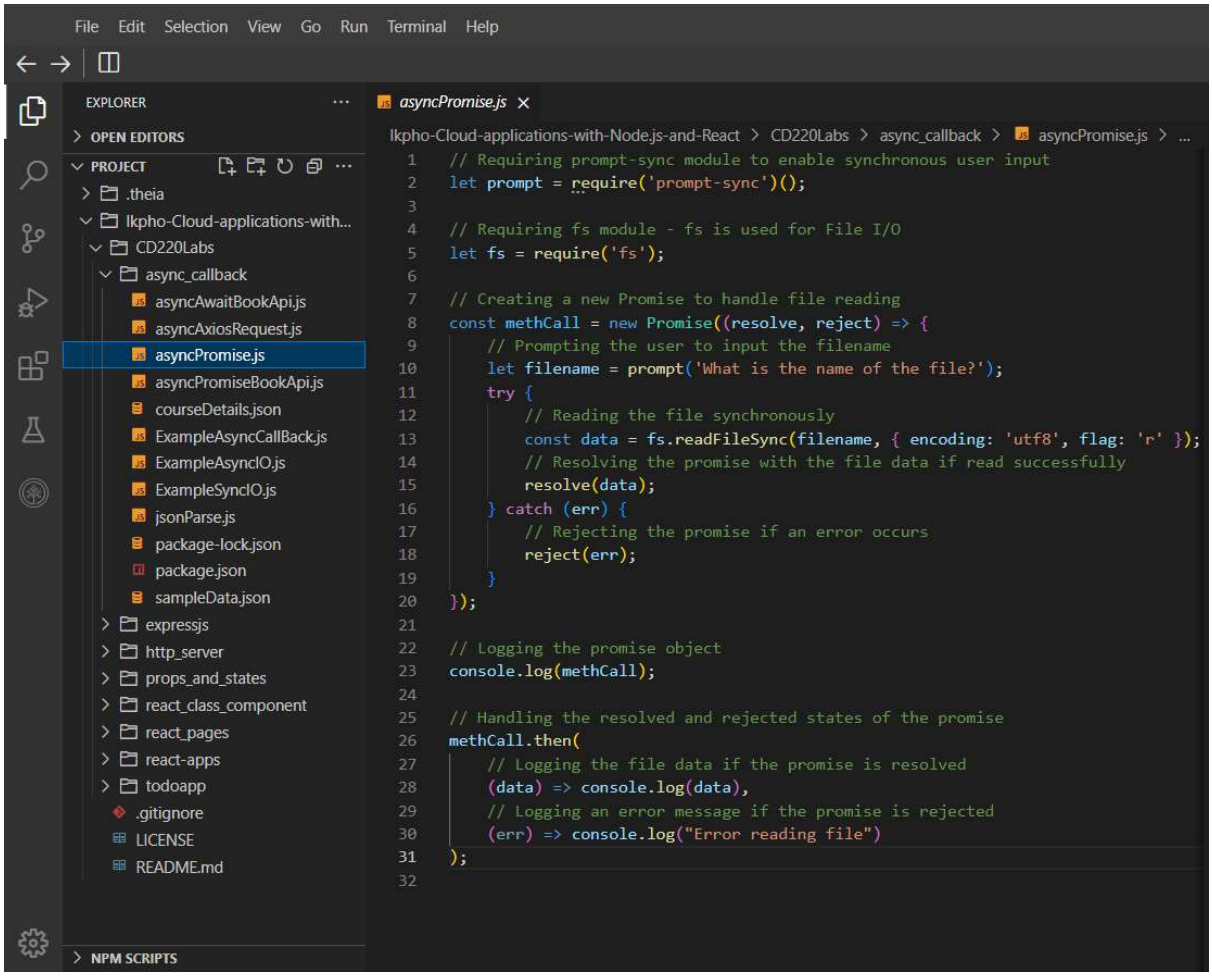
2. En la ventana del terminal, ejecuta el servidor con el siguiente comando.

```
node ExampleAsyncCallBack.js
```

setTimeout es un método de biblioteca incorporado que te permite pasar un método que necesita ser llamado después de un tiempo de espera, como un parámetro. Aquí, firstCallBackMethod se define y luego se pasa como un parámetro a setTimeOut. Como habrás observado, el método se llamará después de 5 segundos. Esto se llama `callback`.

Tarea 5: Promesas

1. En el explorador de archivos, visualiza asyncPromise.js. Se vería así.



▶ También puedes hacer clic aquí para ver el código

Explicación del Código

- `require('prompt-sync')()` importa el módulo `prompt-sync` para habilitar la entrada de usuario sincrónica. `require('fs')` importa el módulo `fs` integrado de Node.js para funcionalidades relacionadas con el sistema de archivos.

- `const methCall = new Promise((resolve, reject) => { ... })` crea una nueva promesa para manejar el proceso de lectura de archivos. La promesa toma una función de callback con los parámetros `resolve` y `reject`.

- Dentro de la promesa:

    - `let filename = prompt('¿Cuál es el nombre del archivo?')` solicita al usuario que ingrese el nombre del archivo.
    - `fs.readFileSync(filename, { encoding: 'utf8', flag: 'r' })` intenta leer el archivo especificado de manera sincrónica con codificación UTF-8.
    - Si el archivo se lee con éxito, se llama a `resolve(data)` con los datos del archivo, resolviendo la promesa.
    - Si ocurre un error, se llama a `reject(err)` con el error, rechazando la promesa.

- `console.log(methCall)` registra el objeto de la promesa.

- `methCall.then(...).catch(...)` maneja la promesa:

    - `then((data) => console.log(data))` registra los datos del archivo si la promesa se resuelve.
    - `catch((err) => console.log("Error al leer el archivo"))` registra un mensaje de error si la promesa es rechazada.

2. En la ventana del terminal, ejecuta el siguiente comando para instalar prompt-sync. Usar `--save` asegura que el archivo package.json se actualice para las dependencias.

```
npm install --save prompt-sync
```

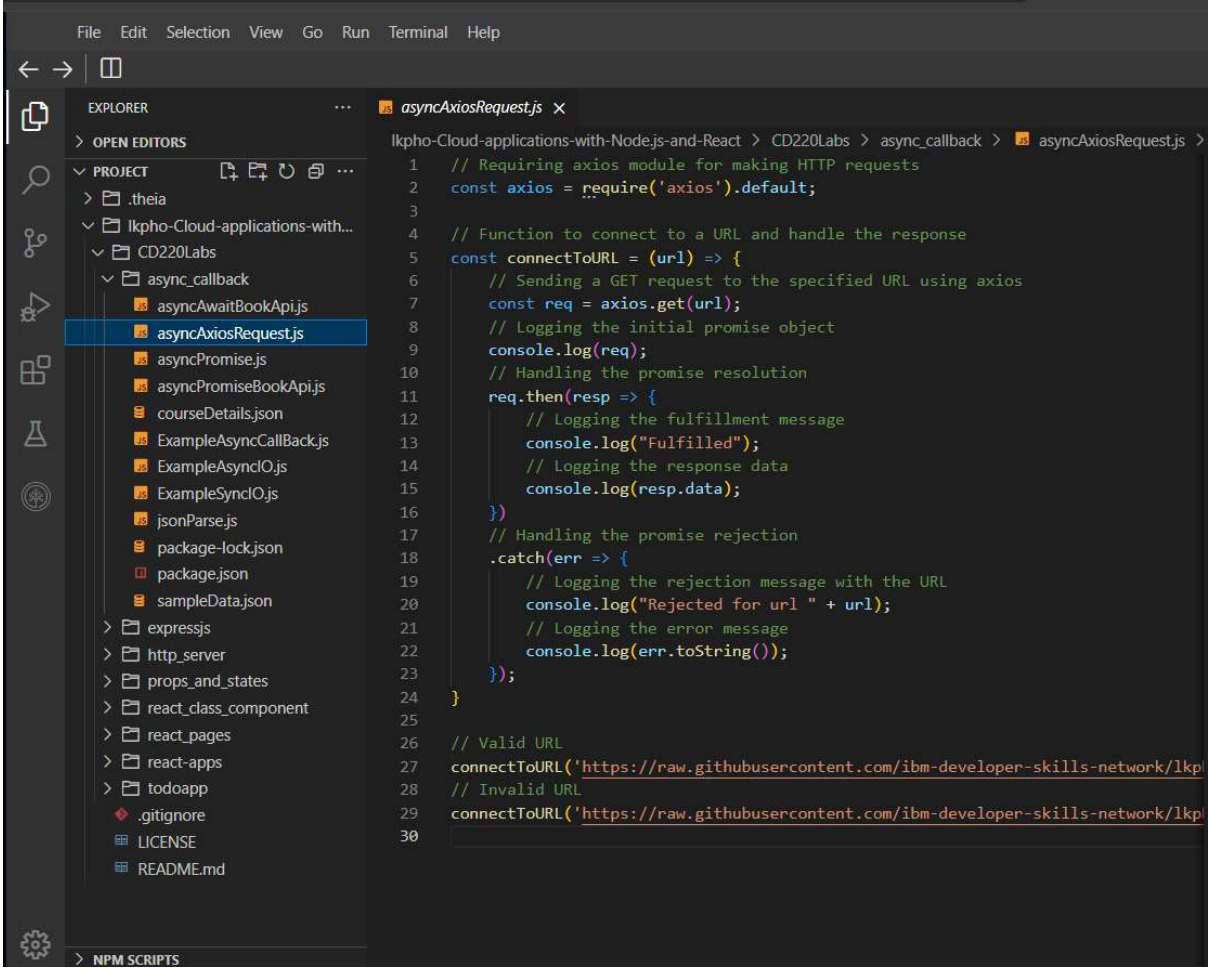
3. En la ventana del terminal, ejecuta el servidor con el siguiente comando. Te pedirá un nombre de archivo. Ingresa un nombre de archivo válido del directorio actual.

```
node asyncPromise.js
```

methCall aquí es un objeto de promesa. Cuando la promesa se cumple, se imprimirá el registro en la consola. Ejecuta el comando anterior nuevamente e intenta pasar un nombre de archivo no válido. Observa cómo se imprime el registro en la consola mientras la promesa es rechazada.

Tarea 6: AsyncAxiosRequest

1. En la vista del explorador de archivos asyncAxiosRequest.js. Aparecería así.



▶ También puedes hacer clic aquí para ver el código

Explicación del Código

- `require('axios').default` importa el módulo `axios`, que se utiliza para realizar solicitudes HTTP.
- `connectToURL(url)` define una función que toma una URL como parámetro y envía una solicitud GET a esa URL utilizando `axios.get(url)`. Registra el objeto de promesa inicial devuelto por `axios.get`.
- Dentro de la función `connectToURL`:
  - `req.then(resp => { ... })` maneja la resolución de la promesa registrando “Cumplido” y los datos de la respuesta (`resp.data`) si la solicitud es exitosa.
  - `req.catch(err => { ... })` maneja el rechazo de la promesa registrando “Rechazado por url” seguido de la URL y el mensaje de error si la solicitud falla.
- La función `connectToURL()` se utiliza para llamar a una URL. Proporcionar una URL válida debería recuperar los datos con éxito, mientras que una URL inválida debería resultar en un error y activar el controlador de rechazo.

2. Para ejecutar este código, necesitamos instalar el paquete axios. Ejecuta el siguiente comando para instalar axios.

```
npm install --save axios
```

3. En la ventana del terminal, ejecuta el código con el siguiente comando.

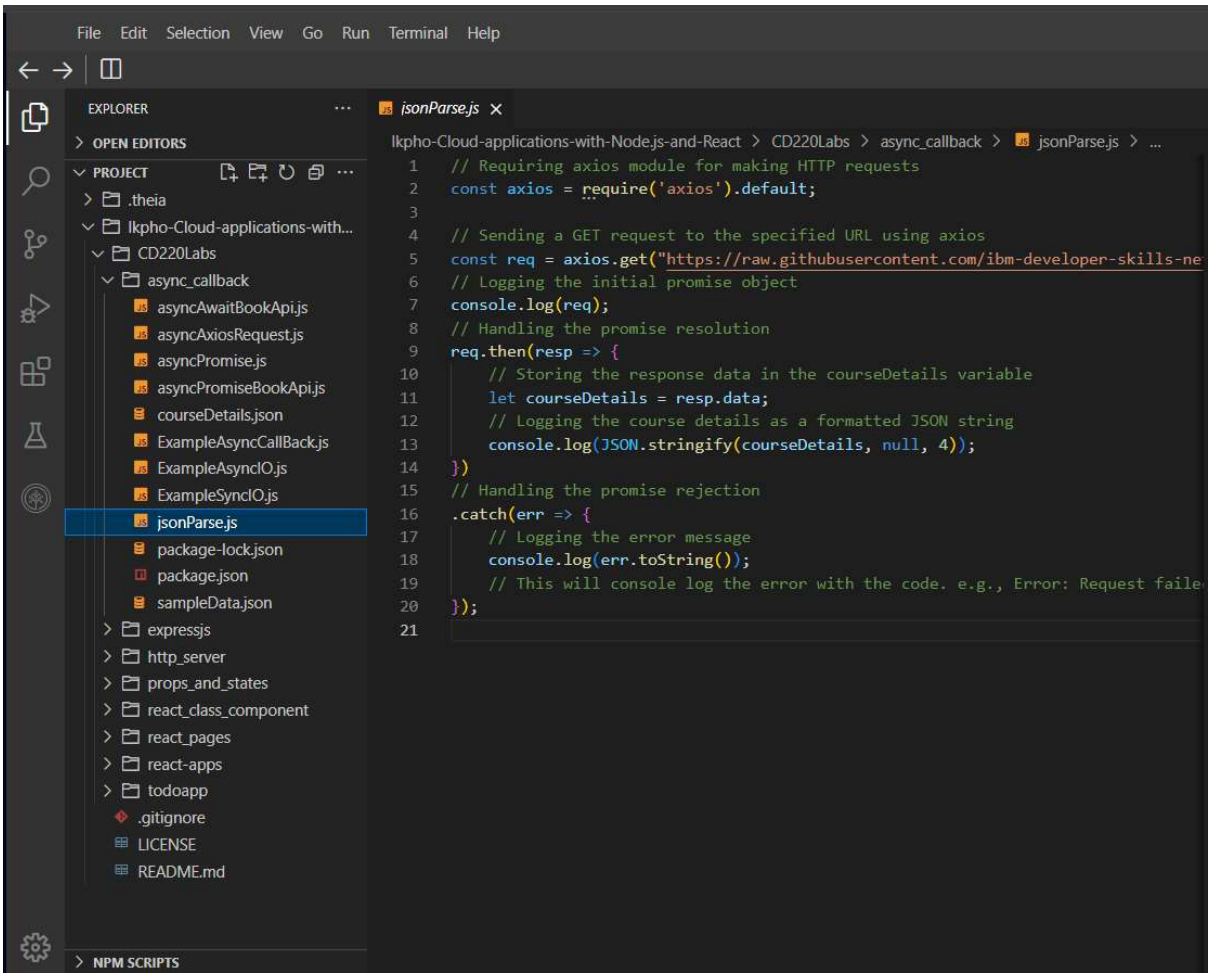
```
node asyncAxiosRequest.js
```

Cuando ejecutas el código, el primer connectToURL es una solicitud axios a una URL válida que devolverá un objeto JSON. El segundo connectToURL es una solicitud axios a una URL no válida. Esto devolverá un mensaje de error apropiado. La salida será como se muestra a continuación.

```
Promise { <pending> }
Promise { <pending> }
Rejected for url https://raw.githubusercontent.com/ibm-developer-skills-network/lkpho-Cloud-applications-with-No
de.js-and-React/master/CD220Labs/async_callback/sampleDate.json
Error: Request failed with status code 404
Fulfilled
{ name: 'Jason', age: '25', department: 'IT' }
```

Tarea 7: Trabajando con JSON

1. En el explorador de archivos, visualiza jsonParse.js. Aparecería así.



▶ También puedes hacer clic aquí para ver el código

Explicación del Código

- `require('axios').default` importa el módulo `axios`, que se utiliza para realizar solicitudes HTTP.
- `axios.get()` envía una solicitud GET a la URL especificada para obtener el archivo `courseDetails.json`.
- La constante `req` almacena la promesa devuelta por `axios.get`.
- `console.log(req)` registra el objeto de promesa inicial en la consola.
- `req.then(resp => { ... })` maneja la resolución de la promesa:
    - `let courseDetails = resp.data` almacena los datos de la respuesta en la variable `courseDetails`.
    - `console.log(JSON.stringify(courseDetails, null, 4))` registra los detalles del curso como una cadena JSON formateada con sangrías.
- `req.catch(err => { ... })` maneja el rechazo de la promesa:
    - `console.log(err.toString())` registra el mensaje de error, que incluye el código de error y la descripción si la solicitud falla (por ejemplo, `Error: Request failed with status code 404`).

2. Para ejecutar este código, necesitamos instalar el paquete axios. Tendrás el módulo axios que instalaste en el ejercicio anterior. Si no, ejecuta el siguiente comando para instalar axios.

```
npm install --save axios
```

3. En la ventana del terminal, ejecuta el código con el siguiente comando.

```
node jsonParse.js
```

Cuando ejecutas el código, se realiza una solicitud axios a una URL remota que devuelve un objeto JSON. Este objeto JSON se convierte en una cadena (o se formatea en una forma legible) y se registra en la consola. La salida será como se muestra a continuación.

```
Promise { <pending> }
{
    "course": {
        "name": "Cloud Application Development",
        "modules": {
            "module1": {
                "name": "Introduction to Server-Side JavaScript",
                "topics": [
                    "Course Intro",
                    "Module Introduction",
                    "Introduction to Node.js",
                    "Introduction to Server-Side Javascript",
                    "Creating a Web Server with Node.js",
                    "Working with Node.js Modules",
                    "Lab - Introduction to Server-side JavaScript",
                    "Module Summary",
                    "Practice Quiz",
                    "Graded Quiz"
                ]
            },
            "module2": {
                "name": "Asynchronous IO with Callback Programming",
                "topics": [
                    "Module Introduction",
                    "Asynchronous I/O with Callback Programming",
                    "Creating Callback Functions",
                    "Promises",
                    "Working with JSON",
                    "Lab",
                    "Module Summary",
                    "Practice Quiz",
                    "Graded Quiz"
                ]
            },
            "module3": {
                "name": "Express Web Application Framework",
                "topics": [
                    "Module Introduction",
                    "Extending Node.js",
                    "Express Web Application Framework",
                    "Your first Express Web Application",
                    "Routing, Middleware, and Templating",
                    "Lab",
                    "Module Summary",
                    "Practice Quiz",
                    "Graded Quiz"
                ]
            },
            "module4": {
                "name": "Building a Rich Front-End Application using REACT & ES6",
                "topics": [
                    "Module Introduction",
                    "Introduction to ES6",
                    "Introduction to Front End Frameworks and React.JS",
                    "Working with React Components",
                    "Passing Data and States between Components",
                    "Connecting React App to External Services (WIP)",
                    "Lab",
                    "Module Summary",
                    "Practice Quiz",
                    "Graded Quiz"
                ]
            }
        }
    }
}
```

¡Felicidades! Has completado el laboratorio del segundo módulo de este curso.

Resumen

Ahora que has aprendido a usar la programación con Callback Asíncrono, iremos más allá y extenderemos las capacidades de nuestro lado del servidor.

Author(s)

Lavanya

© IBM Corporation. Todos los derechos reservados.